

Python Functions

Complete Cheat Sheet · From Root to Advanced

1. WHAT IS A FUNCTION?

A function is a reusable block of code that performs a specific task. You define it once and call it many times.

- ✓ Avoids code repetition (DRY – Don't Repeat Yourself)
- ✓ Makes code organized and readable
- ✓ Easy to test and debug

► Basic Structure

SYNTAX

```
def function_name(parameters):  
    """Docstring: describes what function does"""  
    # function body  
    return value          # optional
```

EXAMPLE

```
def greet(name):  
    """Returns a greeting message."""  
    return f"Hello, {name}!"  
  
result = greet("Nishant")  
print(result)
```

OUT

Hello, Nishant!

2. PARAMETERS & ARGUMENTS

Parameters = variables listed in function definition. Arguments = actual values passed when calling.

► 2a. Positional Arguments (Order matters)

EXAMPLE

```
def add(a, b):          # a, b are parameters
    return a + b

print(add(3, 7))        # 3 and 7 are arguments
```

OUT

10

► 2b. Default Arguments (Optional values)

Default values are used when the caller doesn't provide that argument.

EXAMPLE

```
def greet(name, msg='Hello'):  # msg has default
    print(f'{msg}, {name}!')

greet('Rahul')                 # uses default
greet('Rahul', 'Good morning') # overrides default
```

OUT

Hello, Rahul!
Good morning, Rahul!

ⓘ Default parameters must always come AFTER non-default ones.

► 2c. Keyword Arguments (Named arguments)

Pass arguments by name — order doesn't matter.

EXAMPLE

```
def info(name, age, city):
    print(f'{name} | {age} | {city}')

# Any order works when you use names
info(age=25, city='Delhi', name='Priya')
```

OUT

Priya | 25 | Delhi

► 2d. *args — Variable Positional Arguments

Use *args when you don't know how many positional arguments will be passed. Inside the function, args is a tuple.

EXAMPLE

```
def total(*args):
    print(type(args))      # <class 'tuple'>
    return sum(args)

print(total(1, 2, 3))
print(total(10, 20, 30, 40, 50))
```

OUT

```
<class 'tuple'>
6
150
```

► 2e. **kwargs — Variable Keyword Arguments

Use **kwargs when you don't know how many keyword arguments will be passed. Inside the function, kwargs is a dictionary.

EXAMPLE

```
def show_info(**kwargs):
    print(type(kwargs))      # <class 'dict'>
    for key, val in kwargs.items():
        print(f'{key}: {val}')

show_info(name='Aman', city='Mumbai', age=28)
```

OUT

```
<class 'dict'>
name: Aman
city: Mumbai
age: 28
```

► 2f. Argument Order Rule (MUST follow)

Syntax	What It Means / Example
1. positional	def func(a, b)
2. default	def func(a, b=10)
3. *args	def func(a, *args)
4. keyword-only	def func(*args, key)
5. **kwargs	def func(*args, **kwargs)

FULL ORDER

```
# Correct combined example:
def demo(pos, default=5, *args, kw_only, **kwargs):
    pass
```

3. RETURN STATEMENT

return sends a value back to the caller. Without return, Python returns None automatically.

► Single Return Value

EXAMPLE

```
def square(n):  
    return n ** 2  
  
x = square(6)  
print(x)          # 36
```

OUT

36

► Multiple Return Values (returns a tuple)

EXAMPLE

```
def min_max(nums):  
    return min(nums), max(nums)    # returns tuple  
  
lo, hi = min_max([3, 1, 9, 5])  
print(lo, hi)
```

OUT

1 9

► Early Return (Guard Clause)

EXAMPLE

```
def safe_divide(a, b):  
    if b == 0:  
        return None        # exit early  
    return a / b  
  
print(safe_divide(10, 2))  
print(safe_divide(10, 0))
```

OUT

5.0
None

🔗 A function STOPS executing as soon as it hits a return statement.

4. SCOPE — LEGB RULE

Scope = where a variable is accessible. Python uses LEGB order to look up names.

Syntax	What It Means / Example
L — Local	Inside the current function

E – Enclosing	Inside outer (enclosing) function (nested functions)
G – Global	Module-level variables
B – Built-in	Python's built-in names like print, len, range...

► Local vs Global

EXAMPLE

```
x = 10          # Global

def show():
    x = 99      # Local – does NOT change global x
    print(x)    # 99

show()
print(x)        # 10 – global unchanged
```

OUT

99
10

► global keyword — modify global inside function

EXAMPLE

```
count = 0

def increment():
    global count    # tells Python: use global count
    count += 1

increment()
increment()
print(count)    # 2
```

OUT

2

► nonlocal keyword — modify enclosing variable

EXAMPLE

```
def outer():
    n = 0
    def inner():
        nonlocal n    # refers to outer's n
        n += 1
    inner()
    inner()
    print(n)    # 2

outer()
```

OUT

2

🔗 Avoid global variables — they make code hard to debug. Prefer passing values as arguments.

⚡ 5. LAMBDA FUNCTIONS

Anonymous (nameless) single-expression functions. Best for short, throwaway operations.

SYNTAX

```
lambda parameters: expression
```

► Basic Lambda

EXAMPLE

```
square = lambda x: x ** 2
print(square(5))          # 25

add = lambda a, b: a + b
print(add(3, 4))          # 7
```

OUT

```
25
7
```

► Lambda with sorted(), map(), filter()

EXAMPLE

```
# sorted with key
names = ['Priya', 'Aman', 'Raj', 'Deepika']
print(sorted(names, key=lambda x: len(x)))

# map – apply to each element
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)

# filter – keep elements where condition is True
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)
```

OUT

```
['Raj', 'Aman', 'Priya', 'Deepika']
[2, 4, 6, 8]
[2, 4]
```

📖 Lambda = def without a name, just one expression. For multi-line logic, use a regular def.

📖 6. NESTED FUNCTIONS & CLOSURES

A function defined inside another function. The inner function can remember the enclosing scope's variables even after the outer function has finished.

► Nested Function

EXAMPLE

```
def outer(msg):
    def inner():          # nested inside outer
        print(msg)       # accesses outer's variable
    inner()

outer('Hello from nested!')
```

OUT

Hello from nested!

► Closure — Inner function returned and remembers enclosing state

EXAMPLE

```
def multiplier(factor):
    def multiply(n):
        return n * factor    # factor 'closed over'
    return multiply          # returns the function itself

double = multiplier(2)
triple = multiplier(3)

print(double(10))    # 20
print(triple(10))    # 30
```

OUT

20
30

📖 Closures are the foundation of decorators. The inner function 'closes over' the outer variable.

7. DECORATORS

A decorator wraps another function to add behavior before/after it — without modifying the original function's code.

► Building a Decorator from Scratch

EXAMPLE

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before function runs')
        result = func(*args, **kwargs)  # call original
        print('After function runs')
        return result
    return wrapper

@my_decorator          # sugar for: greet = my_decorator(greet)
def greet(name):
    print(f'Hello, {name}!')

greet('Nishant')
```

OUT

```
Before function runs
Hello, Nishant!
After function runs
```

► Real-world: Timer Decorator

EXAMPLE

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} took {end-start:.4f}s')
        return result
    return wrapper

@timer
def slow_task():
    time.sleep(0.5)

slow_task()
```

OUT

```
slow_task took 0.5001s
```

► functools.wraps — Preserve original function metadata

BEST PRACTICE

```
from functools import wraps

def my_dec(func):
```

```
@wraps(func)          # preserves __name__, __doc__ etc.
def wrapper(*args, **kwargs):
    return func(*args, **kwargs)
return wrapper
```

Always use `@functools.wraps(func)` inside your decorator — it keeps debugging clean.

8. GENERATOR FUNCTIONS

A generator uses `yield` instead of `return`. It produces values one at a time (lazy evaluation) — saves memory for large data.

► Basic Generator

EXAMPLE

```
def count_up(n):
    i = 1
    while i <= n:
        yield i      # pauses here, returns i
        i += 1

gen = count_up(4)
print(next(gen))    # 1
print(next(gen))    # 2
for val in count_up(4):
    print(val, end=' ')
```

OUT

```
1
2
1 2 3 4
```

► Generator vs List — Memory Comparison

MEMORY COMPARISON

```
# List: loads all 10 million numbers into memory
big_list = [x * 2 for x in range(10_000_000)]

# Generator: creates one number at a time
big_gen = (x * 2 for x in range(10_000_000))

import sys
print(sys.getsizeof(big_list))    # ~80MB
print(sys.getsizeof(big_gen))    # ~112 bytes
```

OUT

```
~80,000,056 bytes
~112 bytes
```

Use generators when processing large files or streams — read line by line, not all at once.

9. RECURSION

A function that calls itself. Every recursive function needs a base case (stopping condition) to avoid infinite loops.

► Factorial (Classic Recursion)

EXAMPLE

```
def factorial(n):
    if n == 0:                # BASE CASE – stop here
        return 1
    return n * factorial(n - 1) # RECURSIVE CALL

print(factorial(5)) # 5*4*3*2*1 = 120
```

OUT

120

► Fibonacci (Two recursive calls)

EXAMPLE

```
def fib(n):
    if n <= 1:                # Base cases: fib(0)=0, fib(1)=1
        return n
    return fib(n-1) + fib(n-2)

for i in range(8):
    print(fib(i), end=' ')
```

OUT

0 1 1 2 3 5 8 13

Python's default recursion limit is 1000 (sys.setrecursionlimit). For deep recursion, prefer iteration.

10. HIGHER-ORDER FUNCTIONS

Functions that take other functions as arguments or return functions. Python treats functions as first-class objects.

► map() — Transform each element

EXAMPLE

```
nums = [1, 2, 3, 4, 5]

# Using lambda
squares = list(map(lambda x: x**2, nums))
print(squares)

# Using regular function
def cube(x): return x**3
cubes = list(map(cube, nums))
print(cubes)
```

OUT

[1, 4, 9, 16, 25]

```
[1, 8, 27, 64, 125]
```

► filter() — Keep matching elements

EXAMPLE

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

evens    = list(filter(lambda x: x % 2 == 0, nums))
gt_five  = list(filter(lambda x: x > 5, nums))

print(evens)  # [2, 4, 6, 8]
print(gt_five) # [6, 7, 8]
```

OUT

```
[2, 4, 6, 8]
[6, 7, 8]
```

► reduce() — Accumulate to single value

EXAMPLE

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

total    = reduce(lambda a, b: a + b, nums)    # 15
product  = reduce(lambda a, b: a * b, nums)    # 120

print(total, product)
```

OUT

```
15 120
```

► Functions accepting functions (Custom Higher-Order)

EXAMPLE

```
def apply_twice(func, value):
    return func(func(value))

def double(x): return x * 2

print(apply_twice(double, 3))    # double(double(3)) = 12
print(apply_twice(lambda x: x+5, 10))    # 20
```

OUT

```
12
20
```

11. DOCSTRINGS & TYPE HINTS

► Docstrings — Document your function

EXAMPLE

```
def add(a, b):
    """
    Add two numbers and return the result.

    Args:
        a (int): First number
        b (int): Second number

    Returns:
        int: Sum of a and b

    Example:
        >>> add(2, 3)
        5
    """
    return a + b

help(add)          # prints the docstring
print(add.__doc__) # access docstring as string
```

► Type Hints — Clarify expected types (Python 3.5+)

EXAMPLE

```
def greet(name: str, times: int = 1) -> str:
    return (name + ' ') * times

def process(data: list[int]) -> dict[str, int]:
    return {'sum': sum(data), 'count': len(data)}

# Type hints are NOT enforced – they are documentation
# Use mypy tool to check types statically
```

🔗 Type hints make code self-documenting and enable IDE auto-complete and static analysis.

12. SPECIAL CONCEPTS

► First-Class Functions — Functions as objects

EXAMPLE

```
def shout(text): return text.upper()
def whisper(text): return text.lower()

def speak(func, msg):
    print(func(msg))          # pass function as argument
```

```
speak(shout, 'Hello')      # HELLO
speak(whisper, 'HELLO')   # hello

funcs = [shout, whisper]  # store in a list
for f in funcs:
    print(f('Python'))
```

OUT

```
HELLO
hello
PYTHON
python
```

► Mutable Default Argument Bug (Common Gotcha!)

BUG vs FIX

```
# BUG – list is created ONCE, shared across all calls
def append_bad(item, lst=[]):
    lst.append(item)
    return lst
```

```
print(append_bad(1))  # [1]
print(append_bad(2))  # [1, 2] ← BUG! not [2]
```

```
# FIX – use None as default, create new list inside
def append_good(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

```
print(append_good(1)) # [1]
print(append_good(2)) # [2] ✓
```

OUT

```
[1]
[1, 2] ← BUG
---
[1]
[2] ✓ Fixed
```

🔔 NEVER use mutable objects (list, dict, set) as default arguments. Use None instead.

📖 13. QUICK REFERENCE — ALL CONCEPTS AT A GLANCE

Concept	Syntax / Keyword	Key Point
Define function	<code>def name(params):</code>	Colon + indent body
Return value	<code>return value</code>	Exits function, sends value
No return	(omit <code>return</code>)	Returns <code>None</code> by default
Positional args	<code>func(a, b)</code>	Order matters
Default arg	<code>def f(x, y=10)</code>	<code>y</code> is optional
Keyword arg	<code>func(y=5, x=2)</code>	Order doesn't matter
Variable positional	<code>def f(*args)</code>	<code>args</code> = tuple
Variable keyword	<code>def f(**kwargs)</code>	<code>kwargs</code> = dict
Lambda	<code>lambda x: x*2</code>	Single expression only
Scope — local	<code>x = 5</code> inside <code>def</code>	Not visible outside
Scope — global	<code>global x</code>	Modify global var inside func
Scope — nonlocal	<code>nonlocal x</code>	Modify enclosing var
Nested function	<code>def inner() inside outer()</code>	<code>inner</code> can access outer vars
Closure	<code>return inner_func</code>	Inner func remembers outer state
Decorator	<code>@decorator_name</code>	Wraps function with extra logic
Generator	<code>yield value</code>	Lazy, memory-efficient
Recursion	func calls itself	Must have base case
<code>map()</code>	<code>map(func, iterable)</code>	Transform each element
<code>filter()</code>	<code>filter(func, iterable)</code>	Keep matching elements
<code>reduce()</code>	<code>reduce(func, iterable)</code>	Accumulate to one value
Docstring	<code>"""description"""</code>	First line inside function
Type hint	<code>def f(x: int) -> str:</code>	Documentation, not enforced